

Q1. Implement Entity Relations in Spring Boot using JPA annotations to define database relationships between entities. **@OneToOne** maps a one-to-one relationship where each record in one table corresponds to exactly one record in another. *Example: User and Profile – each user has one profile.*

java

```
@Entity
public class User {
    @Id @GeneratedValue
    private Long id;
    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "profile_id")
    private Profile profile;
}

@Entity
public class Profile {
    @Id @GeneratedValue
    private Long id;
    private String bio;
}
```

Q2. @OneToMany and @ManyToOne represent a bidirectional relationship where one parent has many children, and each child belongs to one parent. **@ManyToOne** is on the child side with `@JoinColumn`, and **@OneToMany(mappedBy="parent")** on the parent side. *Example: Department (one) has many Employees (many).*

java

```
@Entity
public class Department {
    @Id @GeneratedValue
    private Long id;
    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}

@Entity
public class Employee {
    @Id @GeneratedValue
    private Long id;
    @ManyToOne
```

```
@JoinColumn(name = "dept_id")
private Department department;
}
```

Q3. @ManyToMany maps a many-to-many relationship where multiple records in one table relate to multiple records in another, using a join table. *Example: Students and Courses – a student can enroll in many courses, and a course can have many students.*

java

```
@Entity
public class Student {
    @Id @GeneratedValue
    private Long id;
    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses;
}

@Entity
public class Course {
    @Id @GeneratedValue
    private Long id;
    @ManyToMany(mappedBy = "courses")
    private List<Student> students;
}
```